付録A

ドメイン駆動設計の実践：事例研究

この付録では、私がどのようにドメイン駆動設計に取り組むようになったかを紹介します。つまり、あるスタートアップの会社（ここでは仮にマーケットノバス社と呼びます）の物語です。

マーケットノバス社は、設立当初からドメイン駆動設計の手法を採用しました。そして、ドメイン駆動設計に取り組む時にやりがちな、あらゆる失敗を経験しました。さまざまな失敗とやり直しの経験から、多くのことを学びました。ソフトウェア開発を成功させるために、ドメイン駆動設計の考え方とやり方がどういう役割を演じるかの具体例として、マーケットノバス社の失敗とやり直しの物語を紹介します。

この物語は前半と後半に分かれています。前半はマーケットノバス社が取り組んだ五つの区切られた文脈の物語です。どういう判断で文脈を区切り、その結果がどうなったかを紹介します。後半では、マーケットノバス社の事例が、この本の内容にどう反映されているかを説明します。

この物語の説明を始める前に一言。マーケットノバス社は現在はすでに存在していません。つまり、これから説明する内容はマーケットノバス社の宣伝ではありません。もうどこにもない会社なので、何が起きたかをありのままにお伝えできます。

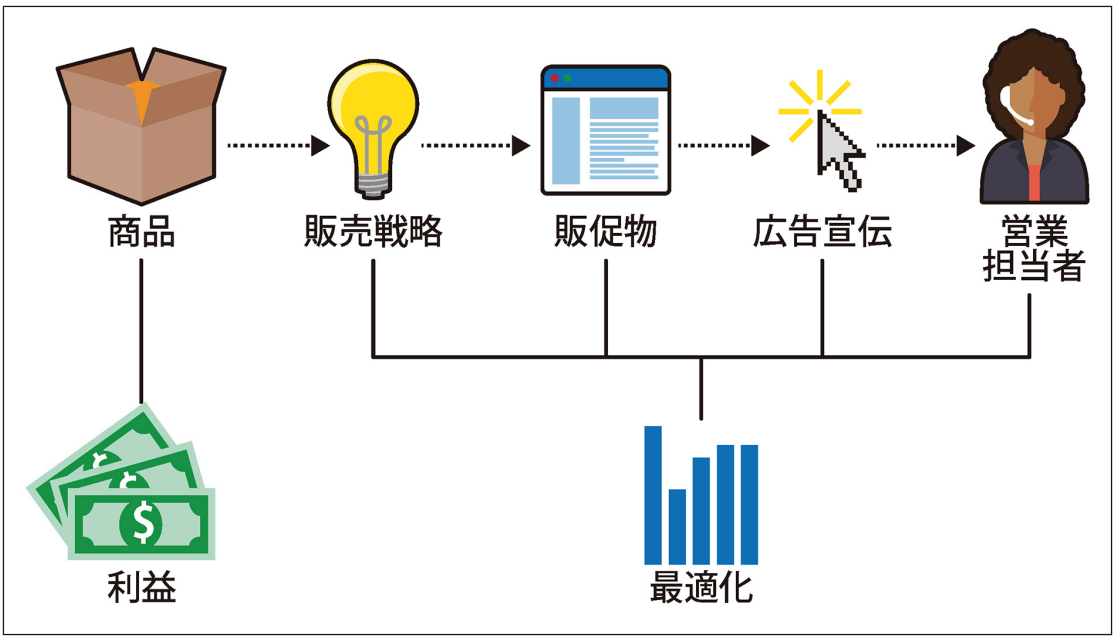
A.1　五つの区切られた文脈

区切られた文脈とその設計の詳細を掘り下げる前に、ドメイン駆動設計の本来のやり方として、マーケットノバス社がどんな事業活動をする会社なのかを

明確にするところから始めます。

A.1.1　マーケットノバス社の事業活動

新製品や新サービスを開発したとして、それをどうやって販売すればよいでしょうか？マーケットノバス社のサービスを利用することで、販売促進に関わるあらゆる活動を外部に委託できます。マーケットノバス社の専門スタッフが販売戦略を立案します。マーケットノバス社のコピーライターやグラフィックデザイナーが販売促進に使うバナーやランディングページを提供します。販売促進活動で獲得した見込み客にマーケットノバス社の営業担当者が接触し、新商品を売り込みます。この一連の活動は図のようになります。



図A-1　販売促進プロセス

もっとも重要な点は、この販売促進プロセスに組み込まれたさまざまな最適化の仕組みです。マーケットノバス社の分析チームがその最適化に責任を持ちます。分析担当者はあらゆるデータを分析し、マーケットノバス社とその顧客に、費用に見合う最高の価値を提供します。もっともうまくいった広告宣伝を見きわめ、もっとも効果的な販売促進用の素材を活用し、もっとも有望な見込み客に営業担当者を優先的に割り当てます。

マーケットノバス社は自己資金だけで設立しました。ですから、資金が底をつく前に、できるだけ短期間で結果を出す必要がありました。そのために、このサービスの最初のバージョンは販売促進プロセスのうち最初の三分の一のステップだけ実装しました。つまり、次の内容です。

- 契約を管理し、外部の広告配信サービスと連係するシステム
- 販促物を作るための素材カタログ
- 販売促進キャンペーンの管理システム

当時の私はこの事業領域の複雑さに圧倒され、その複雑さを整理する方法を模索していました。開発にとりかかってからほどなく、幸運にもすばらしい本に出会いました。そうです。名著『エリック・エヴァンスのドメイン駆動設計』です。

本書の「この本について」を読んだ人なら、エヴァンスの本が、私がずっと探し求めていた答え、つまり業務ロジックの設計と実装のやり方を与えてくれたことをわかっていただけるでしょう。とはいえ、この本を読みこなすのは、当時の私にとって簡単ではありませんでした。それにもかかわらず、戦術的なパターンのところを読んだだけで、私はドメイン駆動設計を完全に理解した気持ちになっていました。

そんな私がどんな設計をしたでしょうか。あのドメイン駆動設計コミュニティの有名人＊1であれば「そうそう、それが正しい」と、すっかり得意がるだろうひどい設計でした。

＊1　@DDD_BoratというTwitter（現X）上のパロディアカウント。ドメイン駆動設計について悪い助言を拡散することで有名。

A.1.2　区切られた文脈 その1：販売促進

私たちの最初の設計スタイルは、言ってみれば「何でも集約」でした。代理店、キャンペーン、広告配置、購入ファネル、媒体など、要求事項にでてくる名詞すべてを集約として表現しました。

そういうすべての集約（らしきもの）を、たった一つの巨大な区切られた文脈にまとめました。そうです。絶対に避けるべき巨大すぎてぞっとする一枚岩のシステムです。

もちろん、この時の私たちの「集約」は本当の集約ではありません。トランザクションの境界と一致せず、業務ロジックをほとんど持っていませんでした。そのかわりに、すべての業務ロジックを巨大なサービス層に記述していました。

ドメインモデルを作ろうとして、結局はアクティブレコードになってしまうのは、貧血ドメインモデルと呼ばれるアンチパターンです。今にして思えば、この設計は「ドメインモデルを作らないやり方」の典型でした。しかし、事業の観点からは、状況はまったく違っていました。

この最初のバージョンは、事業として大成功でした。技術的には問題の多い設計でしたが、私たちは動くソフトウェアを本当に短期間で市場に投入できました。どうしてそれができたと思いますか。

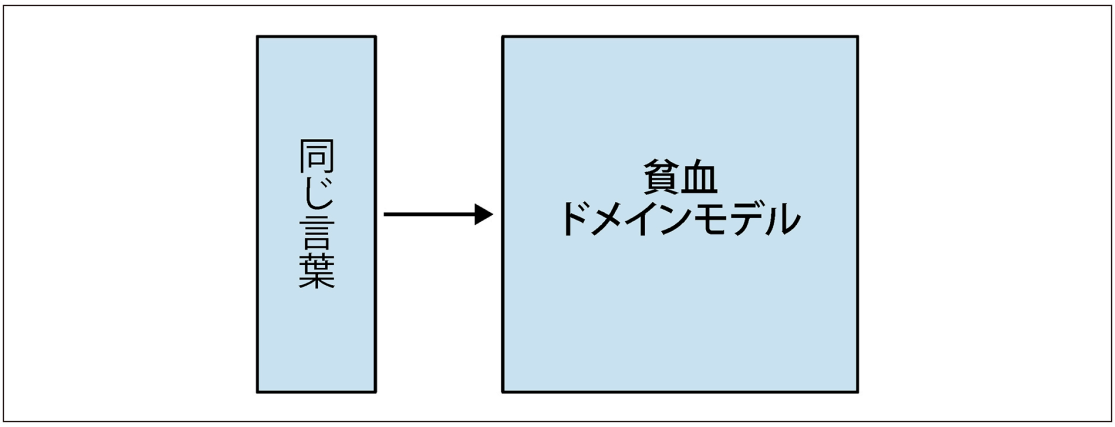
A.1.2.1　短期間で開発できた理由

私たちは、安定した同じ言葉をなんとか手に入れていました。当時の私たちはデジタルマーケティング分野での経験はありませんでした。しかし、この分野に精通した人たちとしっかりと意思疎通ができていました。私たち技術者は彼らの言っていることを理解し、私たちの言っていることを彼らが理解し、びっくりするくらい彼らとはよい仲間になりました。私たち技術者が本気で彼らの知識と経験を学ぼうとしていることを、彼らは心から喜んでくれました。

デジタルマーケティングに精通している人たちと円滑に意思疎通ができたことで、この事業領域の特徴をごく短期間で学び、業務ロジックをすぐさま実装できました。確かに巨大な一枚岩のシステムでしたが、たった二人の技術者が、倉庫みたいな場所で開発するには、その設計で十分でした。繰り返しになりますが、私たちは動くソフトウェアをきわめて短期間で市場に投入できたのです。

A.1.2.2　ドメイン駆動設計の最初の理解

この段階では、私たちはドメイン駆動設計を図A‐2のように理解していました。



図A-2　ドメイン駆動設計の最初の理解

A.1.3　区切られた文脈 その2：顧客管理

販売促進キャンペーンを管理するサービスをリリースしてからわずかな期間で、多くの見込み客を獲得できました。そして、私たちはとても忙しくなりました。販売担当者が見込み客と商談の進捗状況を管理するために、しっかりとした顧客管理システム（CRM）が必要になりました。

その顧客管理システムは、すべての見込み客に関する情報を集約し、いくつかの特性ごとに分類し、世界各地の販売窓口に振り分けることが必要でした。また、マーケットノバス社のサービスを利用する企業の社内システムとの連係も必要でした。見込み客と商談状況の変化を利用企業に通知し、獲得した見込み客の情報と利用企業の持つ顧客情報を紐づけることが必要でした。もちろんこの顧客管理システムでは、できるだけ多くの最適化の機会を提供することが必要です。たとえば、販売担当者が有望な見込み客との商談に集中できるようにしたり、担当者の評価や過去の実績にもとづいて見込み客との商談の振り分け方を決めたり、販売報奨金を柔軟に設定できる機能です。

このような機能を持つパッケージ製品がなかったので、私たちは独自の顧客管理システムを開発することにしました。

A.1.3.1　もっともっと「集約」を！

私たちは、引き続き、技術的な手段に焦点を合わせて、顧客管理システムの実装を始めました。この時も、すべての名詞を集約として宣言し、既存の一枚岩のシステムに押し込めました。しかし、今回は、実装を始めた直後から、何か違和感がありました。

私たちは、集約を追加するたびに、集約の名前の先頭に不自然な単語を追加していることに気がつきました。たとえば、`CRMLead`（顧客管理見込み客）と`MarketingLead`（販売促進見込み客）、`MarketingCampaign`（販売促進キャンペーン）と`CRMCampaign`（顧客管理キャンペーン）などです。興味深いことに、業務エキスパートと会話する時は、私たちはこういう不自然な言葉をまったく使いませんでした。単に`Lead`（見込み客）とか`Campaign`（キャンペーン）と言えば、会話の文脈に応じて、それが何を指しているのかを理解してくれました。

その時、私はドメイン駆動設計の区切られた文脈という考え方を、すっかり忘れていたことに気がつきました。エヴァンス本の該当箇所を読み直したところ、私たちが直面していた問題の解決策は文脈を区切ることだと理解できました。つまり、同じ言葉の一貫性を区切られた文脈で保護するという考え方です。さらに言えば、その時すでに、ヴォーン・ヴァーノンが「Effective Aggregate Design（効果的な集約の設計）」（**https://oreil.ly/tJ0pb**）をネット上に公開していました。私たちが集約の設計で犯したまちがいのすべてが、この資料に明確に書いてありました。私たちは集約を単なるデータ構造として扱い、システムのデータ一貫性を保護するための過大な役割を持たせていました。

私たちは、この重要な気づきを反映するために、一歩下がって、顧客管理システムを再設計することにしました。

A.1.3.2　設計のやり直し

巨大な一枚岩のシステムを、二つの区切られた文脈、つまり販売促進と顧客管理に分割することにしました。もちろん、この段階ではマイクロサービスは目指してはいません。同じ言葉を保護するために必要な、最低限のことをしただけです。

しかし、新規の区切られた文脈（顧客管理）で、販売促進システムで犯したまちがいを繰り返すつもりはありませんでした。貧血ドメインモデルはもう必要ありません。私たちは、定義通りの「本当」の集約を使って、ドメインモデルを実現しようとしました。特に、以下の点に注力しました。

- 一つのトランザクションで変更する集約は一つだけにする
- O-Rマッピングツールにまかせるのではなく、トランザクションの範囲を集約で定義する
- サービス層のクラスを徹底的に減量し、すべての業務ロジックを適切な集約に移動する

私たちはこの正しいやり方に情熱的に取り組みました。しかし、適切なドメインモデルを作り出すのがたいへんであることは、ただちに明らかになりました。

販売促進システムに比べ、あらゆることに時間がかかりました。適切なトランザクションの境界を最初から見つけることは、ほとんど不可能でした。私た

ちはモデルを何度も変えながら試してみました。後から思えば、その時は思いもよらなかったモデルが本当は正しいモデルでした。「正しくやる」ことは高くつきます。つまり、多くの時間が必要です。

予定した期日に間に合わないことが、誰の目にも明らかでした。この状況をなんとかするために、経営陣は、私たちを助けるつもりで、一部の機能の実装を…データベース管理チームに割り当てることにしました。

そうです。業務ロジックをストアードプロシージャで実装することにしたのです。

この決定は、状況を完全に悪化させました。SQLが業務ロジックの記述に適した言語ではないことが理由ではありません。本当の問題は、もっと微妙で、もっと根本的なものでした。

A.1.3.3　バベルの塔の再現

この状況は、暗黙の区切られた文脈を生み出すことで、もっとも複雑なビジネス要素、つまり`Lead`（見込み客）を、その暗黙の境界で分断してしまいました。

その結果、どちらのチームも同じビジネス要素に取り組み、密接に関係した機能をお互いにほとんど会話することなく実装を進めました。同じ言葉とは!? 少し頭を冷やしましょう。二つのチームは、それぞれの独自の言葉で、事業活動と業務ルールを記述しました。

お互いの作ったモデルはまったく一致していませんでした。共通理解などどこにもありません。業務知識は重複し、同じ業務ルールが二重に実装されました。ありがたいことに、業務ロジックを変更する時は、必ず両者の間に不一致が生まれました。

言うまでもありませんが、このプロジェクトは予定した期日にはリリースできませんでした。そして欠陥だらけでした。この低品質のソフトウェアのために、私たちのもっともたいせつな資産、つまり本番環境のデータが、何年にもわたって汚染され続けました。

この泥沼から抜け出すたった一つの方法は、`Lead`（見込み客）集約を完全に書き直すことでした。文脈を適切に区切り直し、すべてを書き直すのは数年

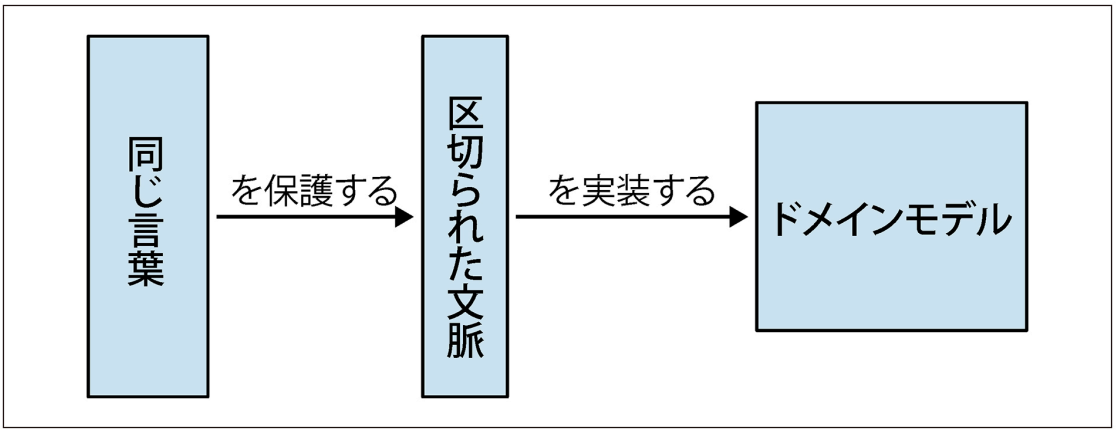
かかりました。簡単なことではありませんでした。しかし本当に欠陥だらけのコードだったために、それ以外の選択肢はありませんでした。

A.1.3.4　ドメイン駆動設計の理解の広がり

事業的には、このプロジェクトは大失敗でした。しかし、この経験から、私たちはドメイン駆動設計をもう少し理解できるようになりました。つまり、以下のことを学びました。

- 同じ言葉を確立する
- 区切られた文脈を使って同じ言葉の完全性を保護する
- あらゆる場所で貧血ドメインモデルを実装するのではなく、あらゆる場所で正しいドメインモデルを実装する

私たちのドメイン駆動設計の理解は、**図A‐3**のようになりました。



図A-3　ドメイン駆動設計の戦略的な考え方を取り入れる

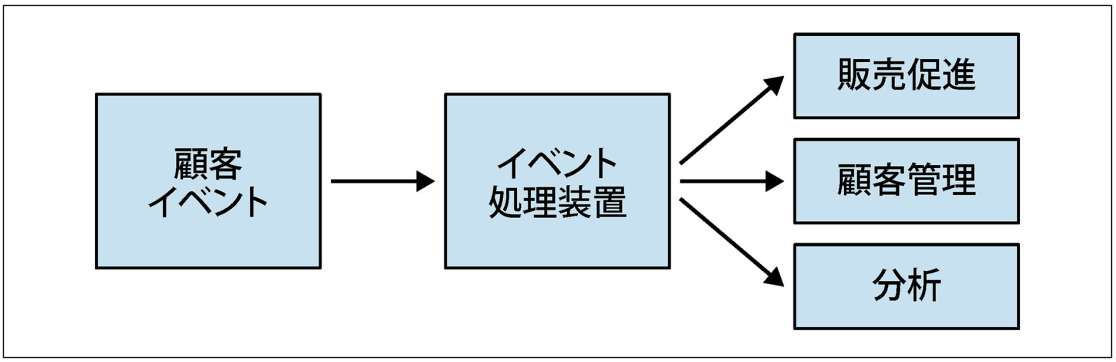
もちろん、この中にはドメイン駆動設計できわめて重要な部分が欠けていました。つまり、業務領域の考え方とそのカテゴリーの判断、そしてそれらがシステムの設計にどう影響するかの知識です。

私たちはできるだけよい仕事をしようとしました。しかし、それは、補完的な業務領域をドメインモデルで実装する、という無駄な努力でした。エリック・エヴァンスが言うように、大規模なシステムのすべてをうまく設計することはできません。私たちはそれを身をもって学びました。そしてその学びを次のプロジェクトに活かそうとしました。

A.1.4　区切られた文脈 その3：イベント処理装置

顧客管理システムの運用を開始した後で、販売促進と顧客管理にまたがった、もう一つの業務領域があるらしいことに気がつきました。外部から到着する顧客イベントの処理を変更する時には、必ず販売促進と顧客管理の両方の区切られた文脈を変更しなければなりませんでした。

概念的には、顧客イベントの処理は販売促進と顧客管理のどちらか一方の責任ではありません。ですから、私たちは、イベント処理のロジックを独立した文脈（イベント処理装置）として切り出すことにしました（図A‐4）。



図A-4　到着する大量の顧客イベントを処理するための区切られた文脈

単純なデータ移動から事業収益は生まれません。かといって、この機能を実現できる適切な製品やクラウドサービスは見つかりませんでした。ですので、私たちは、補完的な業務領域としてイベント処理装置を設計することにしました。

創造的な仕事ではありませんでした。つまり、レイヤードアーキテクチャで簡単なトランザクションスクリプトを何本か作りました。これでうまくいったように見えました。しかしそれは最初だけでした。

私たちの事業が発展するにつれ、このイベント処理装置にさまざまな機能を追加しました。最初は、ビジネスインテリジェンス（BI）チームが希望するフラグの追加でした。つまり、初回コンタクトを表すフラグ、さまざまなイベント種類ごとの初回フラグ、そして業務的な不変性を示すフラグ、などです。

最終的には、これらの単純なフラグ設定機能が複雑なルールと不変条件を持つ本物の業務ロジックに発展していきました。トランザクションスクリプトとして開発を始めた機能が、本格的な中核の業務領域に発展したのです。

残念なことに、複雑な業務ロジックをトランザクションスクリプトで開発しても、よいことは一つもありません。複雑な業務ロジックに対応するように設計をしていなかったために、とてつもなく大きな泥団子になってしまいました。コードベースの変更がたいへんになり、不具合の発生率もあがっていきました。イベント処理装置の設計がうまくいっていないことは明らかでした。1年後には設計をやり直すことにしました。業務ロジックがあまりにも複雑になり、その複雑さに立ち向かえる方法は、イベントソーシングだけでした。私たちは、イベント処理装置のロジックの実装方法をイベント履歴式ドメインモデルにリファクタリングし、イベント処理装置が発信する業務イベントを他の区切られた文脈が購読するようにしました。

A.1.5　区切られた文脈 その4： 販売報奨金

ある日、営業部門の責任者から、現在は手作業でやっている、単純だが手間のかかる作業を自動化したい、という相談がありました。つまり、販売担当者ごとの販売報奨金の計算です。

この時もまず単純なやり方で始まりました。つまり、月に1回、販売担当者ごとの売上の比率を計算し、管理者に報告書を送るだけです。前の時と同じように、この業務領域が中核かどうかをじっくりと検討しました。そしてこの業務領域は中核ではないと判断しました。何か新しい取り組みではないし、この業務から収益を生み出せるわけでもありません。適切なパッケージ製品かクラウドサービスがあれば、まちがいなくそれを使ったでしょう。中核でもないし、かといってパッケージやサービスを使える一般の業務領域でもありません。私たちはこの機能も補完的な業務領域だと考えました。

私たちは補完的な業務領域として設計しました。つまり、サービス層のロジックがアクティブレコードを使うように設計しました（**図A‐5**）。

図A-5　販売報奨金の設計：アクティブレコード＋レイヤードアーキテクチャ

販売報奨金の計算が自動化されると、驚くことに、社内の誰も彼もが創造的になりました。データ分析の担当者はこのプロセスは最適化できると言い始めました。さまざま比率設定を試し、比率を販売額と価格に結びつけ、異なる目標ごとに柔軟な報奨金を設定し、そして…、という具合です。最初の設計が破綻するまでに、たいした時間はかかりませんでした。

この時も、コードベースは制御不能な大きな泥団子になってしまいました。機能を追加するのが極端にたいへんになり、不具合がいろいろ出始めました。金額を扱う機能の不具合は、事業活動にとって深刻な問題を引き起こします。

A.1.5.1　設計のやり直し

イベント処理装置の開発と同じように、今回も最初の設計ではどうにもならない状況になってしまいました。既存のコードを捨て、ゼロから書き直すことが必要でした。今度の設計はイベント履歴式ドメインモデルです。

イベント処理装置と同様に、最初は補完的な業務領域に分類したステムは、成長するにつれ、しだいに中核の業務領域に変化していきました。つまり、販売報奨金の計算が事業収益に大きな影響を及ぼすようになったのです。しかし、イベント処理装置と販売報奨の二つの区切られた文脈には、決定的な違いがあります。

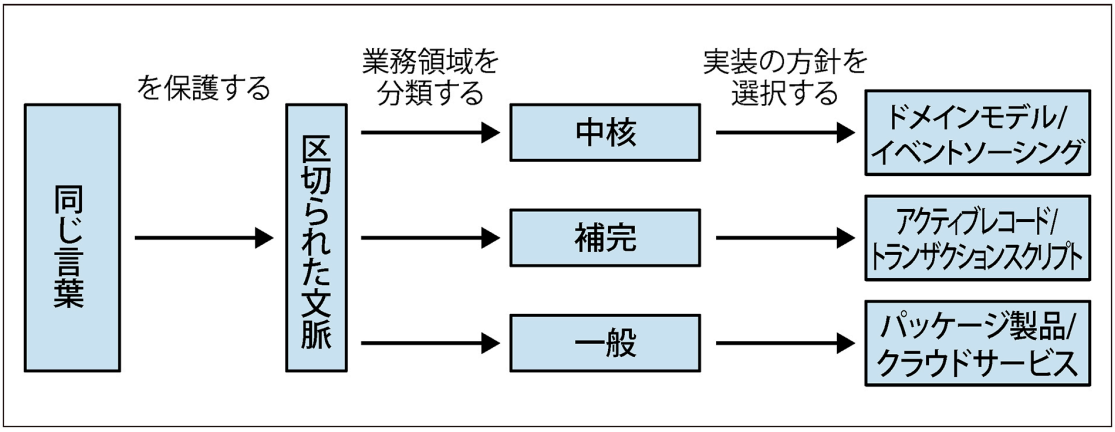
A.1.5.2　同じ言葉

販売報奨金プロジェクトには同じ言葉がありました。アクティブレコードを使った実装でしたが、それでも同じ言葉を使って開発しました。

計算ロジックが複雑になるとともに、業務エキスパートが使う言葉もかなり入り組んできました。ある段階まで複雑になると、もはやアクティブレコードでのモデル化はできなくなりました。この状況が顕在化したことで、設計を根本的に変えるべきことを、イベント処理装置を開発した時よりも早い段階で気づけました。同じ言葉を使っていたことで、丸い穴に四角い棒を打ち込むような無駄な労力と時間を使わずに済みました。

A.1.5.3　ドメイン駆動設計の本格的な理解

この時点で、私たちはドメイン駆動設計の本格的な理解に、ようやくたどりつきました。つまり、同じ言葉、区切られた文脈、業務領域の分類とそれに対応した実装方法の選択です（図A‐6）。



図A-6　ドメイン駆動設計の本格的な理解

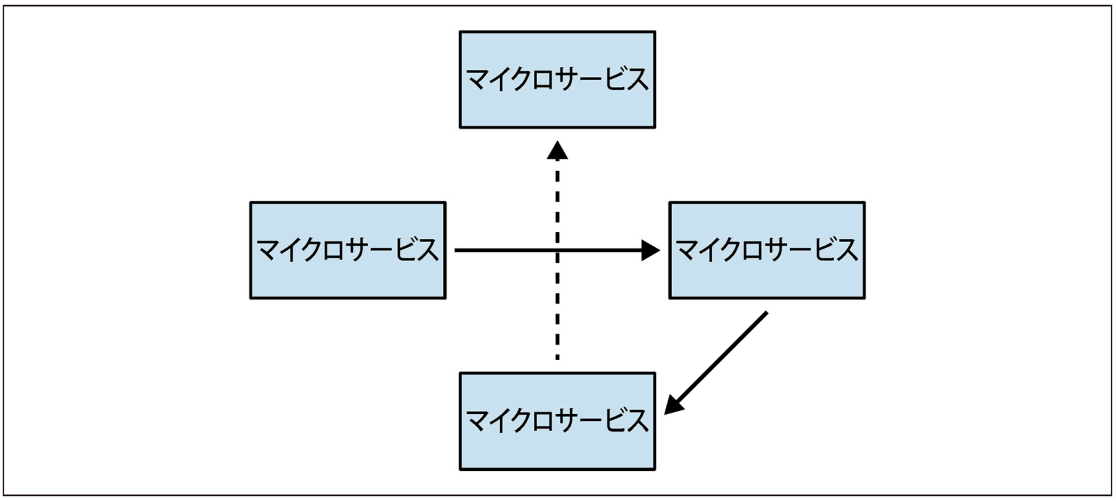
しかし、次のプロジェクトでは、予想外の展開が待っていました。

A.1.6　区切られた文脈 その5：マーケティングハブ

マーケットノバス社の経営陣は、新たな収益源として、それまでとは異なる顧客層へのサービス提供を模索していました。大量の見込み商談を生み出せるマーケットノバス社のサービスを、それまで対象にしていなかった、事業規模の小さい企業に販売することにしました。この取り組みを「マーケティングハブ」と呼びました。

経営陣はこの小規模企業向けのサービスを新たな収益源と考えていました。ですから、新たなサービスは明らかに中核の業務領域です。だとすれば、このサービスを実現するための設計判断は並外れて強力な兵器を投入することでしょう。つまり、イベント履歴式ドメインモデルとCQRSです。そして、当時注目を集め始めていたマイクロサービスにも挑戦することにしました。

私たちの構想は図A‐7のようなものでした。



図A-7　マーケティングハブをマイクロサービスで実現する

それぞれ専用のデータベースを持つ小さなサービス群を同期または非同期の通信で連係させます。紙の上では、完璧な設計に見えました。しかし、実際には、そうではありませんでした。

A.1.6.1　マイクロサービス大失敗

私たちは、浅はかにも、小さなサービスがよいマイクロサービスだと考えました。そして、私たちは集約ごとにサービスを分けました。ドメイン駆動設計の用語で言えば、一つの集約を一つの区切られた文脈としました。

この時も、最初のうちはすばらしい設計に見えました。この設計によって、サービス固有のニーズに合わせて、サービスごとに異なる実装方法を採用できます。特定のサービスをイベントソーシングで実装し、それ以外は状態変更ありの集約で実装できます。そして、それぞれのサービスは独立して修正と拡張が可能です。

しかし、このシステムが成長するに従い、サービス間の通信がどんどん増えていきました。最終的には、ほとんどのサービスは、機能を実現するために

は、さまざまなデータを他のサービスからかき集めることが必要でした。結末は悲惨でした。疎結合のシステムを作ろうとして、実際にできあがったのは、分散型の巨大な一枚岩でした。つまり、地獄のような運用保守作業です。

さらに悪いことに、この設計にはもっと根本的な問題がありました。マーケティングハブの実装方法として、私たちは事業活動をモデリングするために、もっとも複雑な方法を使いました。つまり、ドメインモデルとイベント履歴式ドメインモデルです。私たちは注意深くそれぞれのサービスを作りました。しかし、すべては無駄な努力でした。

A.1.6.2　本当の課題

マーケティングハブの位置づけは中核の業務領域のはずでした。しかし、技術的には複雑なシステムではありませんでした。複雑な実装方法を選択しましたが、実際の業務ロジックは、単純なアクティブレコードとして実装するのが適切な、きわめて単純なものでした。

結局のところ、事業責任者は他社との協業を強化することで利益を得ようとしました。競争優位につながる独自のアルゴリズムをシステムに組み込もうとは考えませんでした。

技術的な複雑さが業務内容の複雑さをはるかに上回っていました。このような複雑さの不一致を「不必要な複雑さ」と呼びます。私たちの設計は、まさにそれでした。明らかに過剰な設計でした。

A.2　ふりかえり

五つの区切られた文脈について説明しました。つまり、販売促進、顧客管理、イベント処理装置、販売報奨金そしてマーケティングハブです。当然のことですが、マーケットノバス社の事業全体では、これ以外にもたくさんの区切られた文脈に取り組むことが必要でした。その中から特に私たちの学びになった五つの区切られた文脈をお伝えしました。

五つの区切られた文脈を順番に説明してきましたが、今度は別の観点から何が起きていたのかを見てみましょう。ドメイン駆動設計の基本的な考え方を、どのように適用できたか、あるいは、どのように適用できなかったか、そしてその結果はどうなったかを見てみましょう。

A.2.1　同じ言葉

私の経験では、同じ言葉を使った開発こそ、ドメイン駆動設計の中核です。業務エキスパートと同じ言葉を使って会話する能力は、私たちに不可欠です。知識を共有する方法として、同じ言葉は、テストやドキュメントよりも、はるかに効果的でした。

さらに言えば、私たちにとって同じ言葉の存在はプロジェクトの成功を予測させる大きな要因でした。つまり、次のような経験です。

- 販売促進システムの開発は、いろいろ課題が多い実装でした。しかし、しっかりした同じ言葉が設計の欠陥を補い、プロジェクトを成功に導きました。
- 顧客管理システムの開発は大失敗でした。気がつかないうちに、私たちは同じ業務領域で二つの異なる言葉を使って開発を進めました。私たちはなんとか適切な設計をしようと努力しましたが、意図の伝達がうまくいかず、大きな混乱だけが残りました。
- イベント処理装置は、単純な補完的な業務領域として開発を始めました。私たちは同じ言葉を使って開発しようとはしませんでした。システムが成長し始めた時、私たちはこの判断を本当に後悔しました。もし同じ言葉を使って開発していれば、もっと短期間でよい成果を生み出せたでしょう。
- 販売報奨金のプロジェクトでは、業務ロジックが桁違いに複雑になりました。しかし同じ言葉を使って開発を進めていたことで、設計方針を変えるべきことに早く気づけました。

このように、同じ言葉は必須の取り組みです。状況によって選択したりしなかったりするものではありません。中核、補完、一般のどの業務領域に取り組む場合でも、必ず同じ言葉を使って開発を進めてください。

できるだけ早い段階から同じ言葉に投資することが重要であることを私たちは学びました。習慣的に使われてきた言い回しを「あるべき言葉」に整えるのは、そうとうな努力とねばり強さが必要です。私たちの顧客管理システムがそうでした。私たちはソースコードを本来の状態に戻すことはできました。簡単なことではありませんでしたが、最終的には、それを達成しました。しかし、「言葉」を変えることは、ソースコードを改善するよりもはるかにたいへんな取り組みです。関係者の何人かが、初期の実装で使った混乱した用語を何年も使い続けたことも事実です。

A.2.2　業務領域

第1章で学んだように、業務領域は中核、補完、一般の三つのカテゴリーに分類できます。そして、ソフトウェアを設計する時は、対象としている業務領域がどのカテゴリーであるかを特定することが重要です。

業務領域のカテゴリーを特定することが難しいこともあります。第1章で検討したように、業務領域を判断する粒度は、開発しているソフトウェアシステムと対応づけることが重要です。たとえば、マーケティングハブは自社の新たな収益源を目指していました。しかし、ソフトウェアシステムとして必要だった機能は、補完的な業務領域でした。収益を得るための優位性、つまり中核の業務領域は、ソフトウェア機能ではなく、他の企業との協力関係と契約条件の交渉力にありました。

さらに言えば、第11章で学んだように、業務領域のカテゴリーを特定するだけでは不十分です。その業務領域が別のカテゴリーに発展する可能性も含めて検討してください。マーケットノバス社では、ほとんどのすべての変更パターンを経験しました。つまり、以下のパターンです。

- イベント処理装置と販売報奨金はどちらも補完的な業務領域として開発を始めました。しかしこれらの業務を最適化する方法を見つけたことで、二つの業務領域はマーケットノバス社の中核の業務領域になりました。
- 販売促進の文脈では、私たちは独自の広告素材カタログを開発しました。類似のサービスと比べて何か特別なことや複雑さはありませんでした。しかし数年たつと私たちが作ったものよりも機能が豊富なオープンソースソフトウェアが登場しました。自社で開発したカタログサービスをこの製品に切り替えることで、業務領域のカテゴリーは補完から一般に変化しました。
- 顧客管理の文脈では、もっとも有望な見込み客を特定する独自のアルゴリズムを実装していました。その最適化機能は改善を続け、別の実装方法にも挑戦したりしました。しかし、最終的には、その最適化機能を機械学習モデルを提供するクラウド上のマネージドサービスに置き換えました。ソフトウェアの設計判断としては、中核から一般への変更でした。
- 何度も説明してきたように、マーケティングハブは中核の業務領域として開発を始めました。しかし最終的には補完的な業務領域のソフトウェアに変わりました。優位性を生み出す源泉は、ソフトウェアとは別の次元に存在しました。

この本で一貫して学んできたように、業務領域のカテゴリーはさまざまな設計判断に影響を与えます。業務領域のカテゴリーの判断が不適切だと、大きな損失につながる可能性があります。イベント処理装置とマーケティングハブがその実例です。

A.2.2.1　設計判断を業務領域と結びつける

マーケットノバス社で私がやっていた、業務領域を楽に特定するためのちょっとしたトリックがあります。それは、業務領域と設計判断の関係の逆転です。まず、業務ロジックの実装方法（トランザクションスクリプト、アクティブレコード、ドメインモデル、イベント履歴式ドメインモデル）を選択してください。憶測や粉飾はしないで、目の前の要求内容に適した実装方法を単純に選んでください。次に、選択した実装方法から業務領域のカテゴリーを判断します。最後に、そうやって判断したカテゴリーが事業方針と整合しているかを確認します。

業務領域と業務ロジックの実装方法の関係を逆転させることで、技術者と業務担当者の間に、新たな対話の機会が生まれます。私たちソフトウェア開発者が業務担当者の知見を必要とするように、業務担当者はソフトウェア開発者の知見を必要とすることもあるのです。

業務担当者がある機能を中核業務だと考えたとしても、その機能を1日で開発できるなら、二つの可能性があります。一つは、開発者が課題を簡単に考えてすぎている場合です。この場合、課題のとらえ方の精度をあげて取り組むことが必要です。もう一つは、その機能が競争優位に結びついていない場合です。この場合、機能の重要性や優先度について再検討が必要です。

別のケースもあります。業務担当者は業務領域を補完的なものと考えているが、業務ロジックの実装は本格的なやり方、つまりドメインモデルやイベント履歴式ドメインモデルが必要な場合です。これはかなり興味深い状況です。

一つの可能性は、業務担当者が必要以上に創造性を発揮して要件を複雑化している場合です。よくあることです。こういう場合、要件はもっと単純にできるはずです。そして単純にすべきでしょう。

もう一つの可能性は、業務担当者がソフトウェア開発の対象となっている業務領域が競争優位の源泉になり得ることに気づけていない場合です。これは販

売報奨金で起きた状況です。開発者がこの不一致を発見することで、すばやく新たな収益源を発見することに貢献できるでしょう。

A.2.2.2　たいへんさを無視しない

業務ロジックの実装がたいへんであれば、それを無視しないでください。そのたいへんさは事業モデルの発展や設計判断の進化につながるきわめて重要な兆候です。設計判断の進化は、業務領域の進化を意味します。業務領域のカテゴリーの判断と業務ロジックの実装方法を再検討すべき状況の変化です。業務領域のカテゴリーが変わったなら、その背景や具体的な状況を知るために、業務エキスパートとよく話し合ってみましょう。業務領域の実情に合わせて業務ロジックの実装方法を変える必要があると判断したら、ひるまずに立ち向かってください。業務ロジックをどのようにモデル化するかが決まり、可能な選択肢をきちんと把握していれば、そのような変化に対応し、業務ロジックの実装方法を洗練された方法に変えることは、かなり簡単にできるようになります。

A.2.3　区切られた文脈の境界

私たちは、マーケットノバス社で、文脈の区切り方をいろいろ試しました。つまり、次の四つです。

- 言葉による境界。具体的には、最初の巨大な一枚岩のシステムを販売促進と顧客管理の二つの文脈に分割し、それぞれの文脈の同じ言葉を保護しました。
- 業務領域ごとの境界。つまり、業務領域ごとに一つの区切られた文脈を実装しました。たとえば、イベント処理装置や販売報奨金です。
- 集約ごとの境界。この方法は、さきほど説明したように、マーケティングハブではあまりうまくいきませんでした。しかし役に立った別の状況はありました。
- 自殺行為となった境界。つまり、顧客管理を開発した時の私たちの失敗です。一つの集約を二つの区切られた文脈に分けてしまいました。こういうことは絶対にやってはいけません。

この四つの区切り方で、どれがおすすめでしょうか。いずれのやり方も、どんな場合でも使えるやり方ではありません。私たちの経験では、最初から小さすぎるサービスに分割して開発するよりは、最初は大きなサービスで開始して、そこから小さなサービスを抽出するほうが安全でした。ですから、私たちは、区切られた文脈も、最初は大きめに区切って開発を始め、業務知識を増や

しながら、後から一部を抜き出して別の文脈に区切り直すようにしました。では初期の区切られた文脈はどのくらいの大きさに区切るのが適切でしょうか。**第11章**で検討したように、事業活動との関連づけがすべてです。つまり、業務知識が貧弱であればあるほど、広い範囲の区切られた文脈で始めます。

私たちはこの経験則でうまくやってきました。たとえば、販売促進と顧客管理は、最初は一つの区切られた文脈で複数の業務領域を対象としました。時間がたつにつれ、初期の大きめの区切られた文脈を分解し、段階的にマイクロサービスに切り出していきました。**第14章**で説明したように、区切られた文脈を発展させていく過程で、常に安全な大きさを維持し続けました。文脈の区切り方のリファクタリングは十分な業務知識を習得した後でやりましょう。そうすることで文脈を小さく区切りすぎる失敗を避けられます。

A.3　まとめ

マーケットノバス社で文脈の区切り方がどう変わっていったかは、私たちがどのようにドメイン駆動設計の理解を深めていったかの物語そのものです（**図A‐6**を参照してください）。

- 私たちはいつでも、業務エキスパートたちと同じ言葉を使うところからスタートしました。対象となる事業活動についてできるだけ多くを学ぶためです。
- モデルが混乱してきたら、私たちは開発範囲を複数の文脈に切り分けました。それぞれの文脈で同じ言葉を使って開発するためです。
- 区切られた文脈が対象とする業務領域のカテゴリー（中核、一般、補完）を特定しました。
- 経験則を使って業務領域ごとに適切な業務ロジックの実装方法を選択しました。
- 業務領域のカテゴリー判断が妥当かどうかを、選択した業務ロジックの実装方法から検証しました。業務領域のカテゴリーと選択した実装方法が一致しなかった場合は、業務担当者と一緒に検討し直しました。この検討作業によって開発プロジェクトの新たな観点をプロダクトの責任者に提供し、その結果、要求が変更されることもありました。
- 業務知識を習得するにつれ、必要であれば、区切られた文脈をさらに小さく切り分けました。

このようなドメイン駆動設計のとらえ方は、私たちの最初の理解と比べると、「いつでもどこでも集約」から「いつでもどこでも同じ言葉」への変化と

言えるでしょう。

マーケットノバス社の創立からの物語を紹介しました。物語の最後としてマーケットノバス社がその後どうなったかをお伝えしておきます。

マーケットノバス社は早い段階で黒字化を達成しました。そして最終的にはこのサービスをもっとも利用していた企業に買収されました。もちろんこの成功はドメイン駆動設計だけが理由ではありません。しかしマーケットノバス社でのソフトウェア開発は常に「スタートアップモード」でした。

私たちの国イスラエルでは「スタートアップモード」とは、他の国であれば「混乱」です。つまり、ビジネス要求は変わり続け、優先度の変更もしょっちゅうです。しかもすべては「できるだけ早く」です。そして研究開発をする余裕はほとんどありません。このような状況で、ドメイン駆動設計を取り入れることで、私たちはさまざまな複雑さに立ち向かい、事業に貢献するソフトウェアを生み出し続けることができました。つまり、振り返ってみれば、ドメイン駆動設計に賭けた私たちは、十分な見返りを手に入れたのです。